DRYDOCK WHITE PAPER SERIES — PAPER 1

# Optimizing Specification-Driven Delivery

Atomic Decomposition, Build Graphs, and Context Engineering for Reproducible LLM Software Delivery

Ed Barlow  Web Cloud Studio  July 2026

# Optimizing Specification-Driven Delivery

**Ed Barlow — Web Cloud Studio**

## Abstract

Specification-driven development (SDD) replaces one-shot prompting with a durable specification from which a large language model builds software. In practice, SDD systems degrade as specifications grow: build quality is a function of prompt size and composition, and a specification large enough to describe a real product is too large to inject into a single build prompt. This paper describes the delivery-optimization layer of Drydock, an open specification-driven methodology. Drydock applies Agile decomposition to reduce an application specification to atomic units of work — stories and spikes with explicit dependencies — and records them in a Manifest, a plain-text graph database that functions as the executable build plan. The build engine walks the dependency graph and assembles, for each step, the minimal context that step requires: the specification files the step implements, compacted derivatives of everything it merely consumes, and shared enterprise stack rules injected once rather than restated per step. Content hashing over applied specifications yields incremental rebuilds: a change to one specification file invalidates only the subgraph that depends on it. The result is a delivery process in which context is engineered rather than hoped for, token cost is estimated and optimized before execution, and an application can be rebuilt — wholly or in optimized chunks — on demand.

**Keywords:** specification-driven development, LLM code generation, context engineering, dependency graphs, Agile decomposition, prompt compaction, reproducible builds

## 1. Introduction

The dominant failure mode of LLM-assisted software delivery is not model capability; it is context management. A modern coding model given a precise, bounded task with exactly the information the task requires performs reliably. The same model given a sprawling prompt — a full product specification, accumulated conversation history, and an open-ended instruction — drifts: it loses constraints stated early in the context, re-implements components it cannot see, and produces software that cannot be regenerated from its inputs [5].

Specification-driven development addresses half of this problem. By making a written specification the source of truth, SDD makes intent durable: the software can be rebuilt by re-executing the specification, and it improves by editing the specification rather than by accreting undocumented conversational patches. Systems such as GitHub's Spec Kit [4] have established the specification artifact as the modern unit of software intent.

SDD as commonly practiced, however, leaves the second half unsolved: there is no *process* between the specification and the build. A specification large enough to define a real application — tens of pages, dozens of features, persistent schemas, shared branding — cannot be handed to a model as a single instruction with any expectation of reproducible output. Quality is related to prompt size and complexity; past a threshold, adding specification text reduces build fidelity rather than increasing it. Working software that cannot be reproduced from its specification is not, in any engineering sense, working software.

This paper presents the optimization layer of **Drydock** [1], a specification-driven delivery methodology whose thesis is that the missing ingredient is process — specifically, the two processes the software industry spent twenty-five years validating: Agile decomposition and test-driven development. A companion paper [2] treats the correctness half of the argument (acceptance criteria and step accuracy). This paper treats the optimization half: how decomposing an Epic into atomic stories, arranging those stories in a dependency graph, and engineering the context of every build step yields delivery that is simultaneously cheaper, more accurate, and reproducible.

The contributions are:

1. **Atomic decomposition as a context-bounding device.** Agile story decomposition is reinterpreted as a partitioning of the specification into units whose build context fits a declared token budget.
2. **The Manifest, a plain-text graph database of work.** Stories, spikes, features, and their dependencies form an executable graph with per-block state, provenance, and evidence.
3. **Context-size-aware file stacking.** Each build prompt is assembled deterministically from typed specification files, with story points estimated in tokens and similar work grouped to amortize shared context.
4. **Compaction and the builder/consumer distinction.** Specification files are reduced to their callable surface for consumers, so that implementation detail is injected exactly once — into the story that builds it.
5. **Incremental, hash-verified rebuilds.** Content hashes over applied specifications localize change, so the application is rebuilt in optimized chunks rather than from scratch.

## 2. Atomic Units of Work

Drydock treats the imported specification corpus as an Agile Epic. The `drydock analyze` command performs Agile decomposition: the LLM, acting in the role of an Agile best-practices team, splits the Epic into features and stories, attaches acceptance criteria to each, and surfaces blockers and open questions for the product owner (in Drydock, the *Commander*) to resolve before planning proceeds [1, §"SAIL Phase 2"].

The decomposition is not stylistic; it is the load-bearing optimization. A story in Drydock is an atomic unit of work with a defined contract:

- **`implements:`** — the typed specification files this story realizes in code;
- **`context:`** — read-only supporting specifications;
- **`depends:`** — the stories or spikes that must be verified before this story may run;
- **`instructions:`** — the bounded task statement;
- acceptance criteria carried in the implemented specification files themselves.

Because every story names its inputs explicitly, the context of a build step is a *computed property of the graph*, not a judgment call made at prompt time. The unit is atomic precisely when its declared context fits within the configured budget (`PROMPT_WARN_TOKENS`, default 50,000 tokens); decomposition continues until every unit satisfies that bound.

Where a question cannot be answered from the specification — a parser selection, an unproven integration — decomposition emits a **spike** rather than a story. A spike's product is a `finding:`, a durable text answer recorded in the graph and available as context to every downstream story. Research is thereby performed once, persisted, and never re-derived inside a build prompt.

Decomposition follows the shape of the system: web applications decompose by route and screen, CLI tools by command, libraries by public API symbol, pipelines by dataset [1, §"Specification Decomposition Methodology"]. Each typed specification file declares `Provides` and `Consumes` interface points, from which the planner computes `Depends On` mechanically. This mirrors the classical criterion for modular decomposition — partition on information-hiding boundaries, not on narrative convenience [6] — applied to specification text rather than code.

## 3. The Manifest: A Graph Database of the Build

`drydock plan` converts the reviewed analysis into typed Blueprint specification files and a single executable artifact, `MANIFEST.md` — a graph build plan expressed as structured Markdown [1, §"The Manifest"]. The Manifest is deliberately a *plain-text graph database*: nodes are `feature`, `story`, `spike`, and acceptance blocks; edges are `depends:` references between stable block identifiers; node attributes carry state (`pending`, `closed/verified`, `closed/failed`), evidence paths, and prompt-assembly fields.

Three properties follow from making the graph the plan of record:

**The runnable frontier.** At any moment, the set of blocks whose external dependencies are all `closed/verified` — the *frontier* — is computable by inspection. `drydock build` executes the frontier; passing acceptance unlocks the next dependent set. Build order is therefore a property of the data, not of an agent's judgment mid-run.

**Estimation in the native currency.** During planning the LLM estimates story points for each story *in tokens* — the actual unit of LLM delivery cost. Token-denominated story points make the classic Agile planning session quantitative: the Commander sees, before execution, what each story costs and where grouping will save.

**Provenance.** The Manifest preamble carries an `applied_specs` registry: one line per specification file that has been applied by a successful build, recording its content hash, commit, applying story, and timestamp. This registry is the foundation of incremental rebuild (§6) and is never regenerated by replanning — it is the durable memory of what has been built from which exact text.

The Manifest is one generated execution view of the Blueprint, not a second product definition. Intent lives in the typed specification files; the graph exists to order, bound, and verify the work of realizing them.

## 4. Order of Operations and Context Grouping

Given the graph, delivery becomes an optimization problem: choose an execution order and a grouping of stories that (a) respects dependencies, (b) keeps every prompt under the token budget, and (c) minimizes total injected context.

Drydock's planner emits a default order following a Foundation → Data and Persistence → Features → User Interface progression, and groups similar stories into build blocks so that shared context — the architecture file, the stack rules, a common feature specification — is injected once per group rather than once per story. Grouping is exposed to the Commander in `BUILD_COMPASS.md`, the story-planning artifact: the Commander may reorder stories so testable foundations land first, regroup work so one agent run covers a set of related stories, or accept the default [1, §"Agile Build Planning"].

The token arithmetic is straightforward but decisive. If $k$ stories each require a shared context of $c$ tokens plus a private specification of $s_i$ tokens, ungrouped execution costs $k \cdot c + \Sigma s_i$ of context injection; grouped execution costs $c + \Sigma s_i$. In enterprise settings $c$ is dominated by exactly the material that never changes between stories — architecture, database contract, stack rules, branding — so grouping routinely removes the majority of duplicate context from a build. The same arithmetic drives the ordering heuristic: build the files that everything else consumes first, so that all later steps can receive their *compact* derivatives rather than their full text (§5).

Each build prompt is then assembled deterministically by file stacking: the Commander's standing intent (`COMPASS.md`), the specification files the block `implements:`, compact derivatives of its `context:`, the applicable stack rules, and the block's instructions — and nothing else. `drydock build --dry-run` prints the assembled file list, prompt size, and token estimate without executing, making context composition itself a reviewable artifact.

## 5. Compaction and the Builder/Consumer Distinction

The single largest source of duplicate context in specification-driven builds is implementation detail injected into steps that only *use* an interface. Drydock names this the **builder/consumer distinction**: the story that builds a feature needs its full specification; every story that merely calls it needs only the callable surface [1, §"Compaction"].

`drydock rigging compact` produces, for any specification file, a `_compact.md` derivative containing only routes, signatures, typed parameters, and one-line summaries — discarding rationale, examples, and internal design. Consumer stories receive the derivative; the one builder story receives the source. The canonical case is the database contract: `DATABASE.md` specifies schemas, migrations, and a typed access-class library, while `DATABASE_compact.md` carries only class names, method signatures, and return types. Only the foundation story that implements the database ever sees the full file; every feature built on top of it sees an interface a fraction of the size.

Two engineering details make compaction safe at scale:

**Compact stability.** Recompaction reproduces the existing derivative verbatim unless the source contains a structural change to the extracted contract. An unchanged derivative keeps its bytes and its `applied_specs` hash, and therefore triggers no rebuild cascade. Cosmetic edits to a source file do not invalidate the portfolio.

**Freshness gating.** A build step that requires an absent or stale compact derivative stops with a directive rather than silently substituting the full file — the context budget is a contract, not a preference.

Compaction generalizes beyond project files to the enterprise layer. **Rigging** — Drydock's portfolio-governance tree of business rules, per-technology stack files, and branding — ships with pre-built compact derivatives. An organization writes its stack guidance once; every project's build steps receive the compacted rules relevant to their technology, and every delivered project conforms to the same

conventions without any per-project restatement. For an enterprise running many targets, this converts stack and branding governance from a per-prompt copy-paste tax into a shared, versioned, deduplicated input.

## 6. Rebuilding in Optimized Chunks

Reproducibility in Drydock is not a slogan about determinism; it is a mechanical consequence of provenance. Before executing any agent, `drydock build` compares every previously applied specification in `applied_specs` against current file content. A hash mismatch blocks the build and names the stale file, its recorded and current hashes, and the remediation path [1, §"drydock build"].

Change therefore localizes along graph edges:

- Editing one `FEATURE-*.md` dirties that file; replanning resets to `pending` exactly the blocks whose `implements:` include it, and preserves the state of every block whose inputs are clean. The next `drydock build` rebuilds the affected subgraph — an optimized chunk — and nothing else.
- Sealed foundational specifications ( `ARCHITECTURE.md` , `DATABASE.md` , `UI-GENERAL.md` ) cannot be casually dirtied; amending them requires an explicit change ticket processed by `drydock refit` , reflecting the true blast radius of foundational change.
- A Commander may deliberately dirty a file to force re-application of its story — the inverse operation, using the same machinery.

Full rebuild is the degenerate case of the same process: with an empty target, the frontier starts at the foundation and the graph replays in order. Because every prompt is assembled from versioned files rather than conversation history, the rebuild consumes no memory of the previous build beyond the Blueprint itself. The specification, the graph, and the hashes *are* the build system — in the same sense that a build graph of object files and header hashes is a compiler's build system. Drydock's claim is that LLM delivery deserves the same discipline `make` brought to compilation: declared inputs, computed order, and rebuilds proportional to change.

## 7. Related Work

Spec Kit [4] established specification artifacts and task lists as the interface to coding agents; a Drydock story is deliberately an enriched Spec Kit task — with states, dependencies, acceptance, and prompt-assembly fields — and Drydock imports Spec Kit projects directly. Agentic coding frameworks orchestrate multi-step builds but typically treat context assembly as emergent from agent memory rather than as a computed property of a declared graph. The context degradation that motivates bounding prompt size is well documented empirically [5]. The decomposition criterion follows Parnas [6]; the process vocabulary is the standard Agile canon [3]. The correctness guarantees that make graph-ordered delivery trustworthy — acceptance criteria declared before build and verified deterministically after it — are the subject of the companion paper [2].

## 8. Conclusion

Specification-driven development made software intent durable but left delivery unmanaged. Drydock's answer is process: decompose the Epic into atomic, token-bounded stories; record them in a plain-text graph database with explicit dependencies and provenance; assemble every build prompt deterministically from typed files; compact what is consumed and inject full detail only where it is built; and let content hashes confine every rebuild to the subgraph a change actually touches. Context is engineered, not hoped for — and software built this way can be rebuilt, chunk by optimized chunk, for as long as its specification lives.

## References


[1] E. Barlow. *Drydock Specification: Agile Specification-Driven Design — The SAIL Methodology for Governed Software Delivery.* Web Cloud Studio, 2026. https://github.com/webcloudstudio/Drydock

[2] E. Barlow. *Guaranteed Step Accuracy: Combining Agile Decomposition and Test-Driven Development in LLM Software Delivery.* Web Cloud Studio, 2026.

[3] K. Beck et al. *Manifesto for Agile Software Development.* 2001. https://agilemanifesto.org

[4] GitHub. *Spec Kit: Toolkit for Spec-Driven Development.* 2025. https://github.com/github/spec-kit

[5] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. "Lost in the Middle: How Language Models Use Long Contexts." *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.

[6] D. L. Parnas. "On the Criteria To Be Used in Decomposing Systems into Modules." *Communications of the ACM*, 15(12):1053–1058, 1972.